7 Spark Execution

How Spark runs your code in parallel and returns the result

PySpark | Apache Spark

1. Overview

- Spark uses lazy evaluation.
- When an action is triggered (e.g., count(), collect()), Spark builds a DAG of operations.
- The DAG is divided into Stages.
- Each Stage is broken into Tasks.
- Tasks are executed in parallel on executors.
- Results are returned to the driver.

Key Point

Spark = Lazy + DAG + Parallel Execution

2. Execution Flow

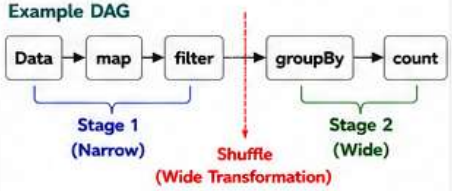
1. **Application / Driver Program**
   You write code and trigger an action.
2. **DAG (Directed Acyclic Graph)**
   Spark creates a DAG of all operations.
3. **Stages**
   DAG is divided into stages based on wide transformations.
4. **Tasks**
   Each stage is divided into small tasks.
5. **Execution on Cluster**
   Tasks are sent to executors and run in parallel.

Result returned to Driver

3. DAG to Stages

- Narrow Transformations (e.g., map, filter) → Rows stay in the same partition.
- Wide Transformations (e.g., groupBy, join, distinct) → Data needs to be shuffled.
- A new Stage is created after each wide transformation.

Example DAG



- map, filter → Narrow → Same Stage (Stage 1)
- groupBy, count → Wide → New Stage (Stage 2)

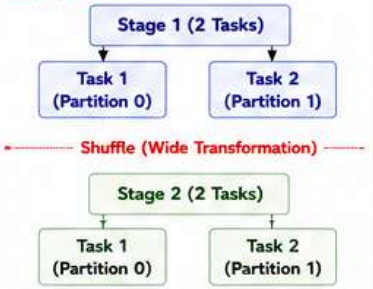
Key Point

Wide Transformation = Shuffle = New Stage

4. Stages and Tasks

- Each Stage is split into multiple Tasks.
- Number of tasks = Number of partitions in that stage.
- Tasks in the same stage can run in parallel.

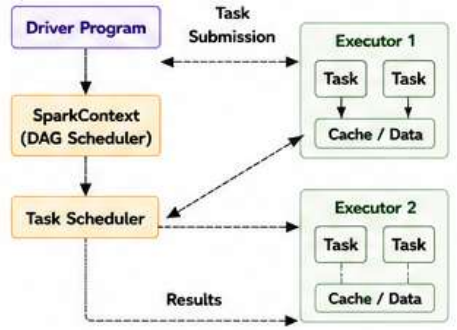
Example



Key Point

More partitions → More tasks → More parallelism

5. Cluster Architecture



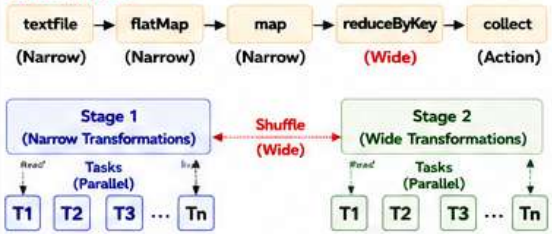
- Driver → Coordinates and schedules tasks
- Executors → Run tasks and store data
- Communication → Driver ↔ Executors

6. Example: Word Count Execution

Code

```
val lines = sc.textFile("file.txt")
val words = lines.flatMap(_.split(" "))
val pairs = words.map(word => (word, 1))
val counts = pairs.reduceByKey(_ + _)
counts.collect()  // Action
```

Execution Flow



7. What Happens During Execution?

1. Driver builds the DAG when action is called.
2. DAG is divided into stages at wide transformations.
3. Each stage is divided into tasks (based on partitions).
4. Tasks are sent to executors.
5. Executors run tasks in parallel and return results.
6. Results are aggregated and returned to driver.

Key Point

Everything is lazy until an action is called.
Spark tries to maximize parallelism.
Data is moved only at wide transformations (shuffle).

8. Narrow vs Wide Transformations

Type	Definition	Examples	Shuffle	Same Stage?
Narrow	Each partition's data is used only by one partition in the next step.	map(), filter(), flatMap(), select(), union()	No	Yes
Wide	Data from one partition is required by multiple partitions in the next step.	groupBy(), reduceByKey(), join(), distinct(), sortBy()	Yes	No

Key Point

Wide transformations are expensive because of data shuffle.

9. Performance Factors

- **Number of Partitions**
  More partitions → More parallelism, but too many → Overhead
- **Wide Transformations**
  Cause data shuffle → Expensive
- **Caching / Persistence**
  Avoids recomputation
- **Cluster Resources**
  More cores & memory → Better performance

10. Quick Revision

- ✓ Spark uses lazy evaluation.
- ✓ Action triggers DAG creation.
- ✓ DAG is divided into stages based on wide transformations.
- ✓ Stages are divided into tasks.
- ✓ Tasks run in parallel on executors.
- ✓ Results are returned to driver.

DAG → Stages → Tasks → Parallel Execution → Results

11. Interview Questions

1. What is lazy evaluation in Spark?
2. What is a DAG in Spark?
3. What is the difference between narrow and wide transformations?
4. When is a new stage created in Spark?
5. How does Spark decide the number of tasks?
6. What happens during a shuffle?
7. What is the role of driver in Spark execution?
8. What is the role of executors?
9. How does Spark achieve parallelism?
10. How can you improve Spark performance?

12. Memory Trick

- **D** – Driver builds DAG
- **A** – Action triggers execution
- **G** – DAG is divided into Stages
- **S** – Stages are split into Tasks
- **T** – Tasks run in parallel
- **R** – Results returned to Driver

DAG to Results!

Spark Execution = Lazy Evaluation + DAG + Stages + Tasks + Parallelism = Fast and Scalable Processing!

8 Spark RDD Lineage

Remembering the Journey of Data to Rebuild when Needed

PySpark | Apache Spark

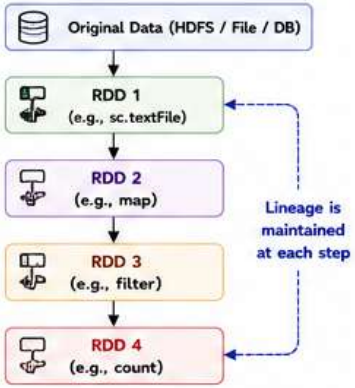
1. What is RDD Lineage?

- Lineage is the history of all transformations applied on an RDD to create it.
- Spark uses lineage to rebuild lost or corrupted partitions.
- RDDs are immutable. Lineage is the key to fault tolerance.
- Each RDD "remembers" how it was created from its parent RDD(s).

Key Point

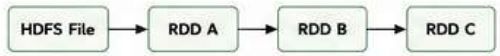
Lineage = The recipe to recreate an RDD from the original data.

2. How Lineage Works?



If any partition of RDD 4 is lost, Spark uses lineage to go back and recompute it from the original data.

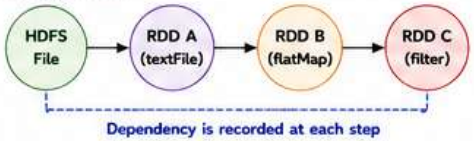
3. Lineage Example



Code:

```
lines = sc.textFile("file.txt")                 # RDD A
words = lines.flatMap(lambda line: line.split())  # RDD B
longWords = words.filter(lambda w:  len(w) > 5)   # RDD C
count = longWords.count()                       # Action
```

Lineage Graph:



Key Point

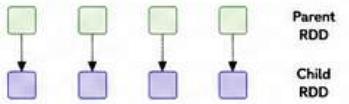
Spark can rebuild any RDD (e.g., RDD C) from its lineage:
HDFS File → RDD A → RDD B → RDD C

4. Types of Dependencies

1) Narrow Dependency (One-to-One)

- Each partition of child RDD depends on a small number (usually one) of partitions of parent RDD.
- Allows pipelining and parallelism.

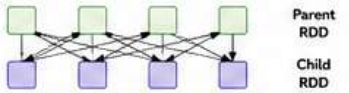
Examples: map(), filter(), flatMap(), union()



2) Wide Dependency (Many-to-Many)

- Each partition of child RDD depends on many partitions of parent RDD.
- Triggers a shuffle.

Examples: groupByKey(), reduceByKey(), join(), distinct(), sortByKey()



5. Why Lineage is Important?

Fault Tolerance
If a partition is lost, Spark rebuilds it using lineage.

Lazy Evaluation
Lineage is built when transformations are defined, not executed.

Scalability
Spark can recompute lost data instead of replicating always.

Debugging
Helps understand how data flows and where issues occur.

Key Point

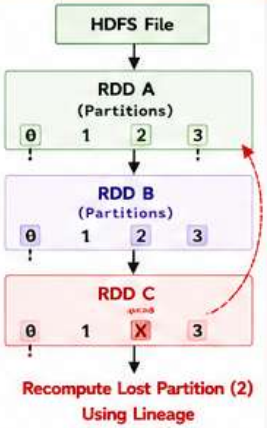
Lineage = Fault tolerance + Scalability + Efficiency

6. Lineage in Action (Partition Loss)

Suppose RDD C, partition 2 is lost.

Spark will:

1. Go back to RDD B (filter) and find which partitions were used.
2. Go back to RDD A (flatMap) and so on...
3. Finally read from original data (HDFS File).
4. Recompute the lost partition and continue.



Key Point

Lineage allows Spark to recover from failures without manual intervention.

7. Lineage vs Checkpoint

Aspect	Lineage	Checkpoint
Definition	History of transformations to recreate RDD	Save RDD to stable storage (HDFS) and truncate lineage
Purpose	Fault tolerance	Manage long-lineage & improve performance
Storage	No extra storage	Requires storage (HDFS)
Overhead	Recompute if data lost	No recomputation (after checkpoint)
When to Use	Default (always)	When lineage is very long or expensive to recompute

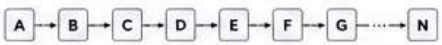
Key Point

Checkpoint breaks the lineage and saves the RDD physically to stable storage.

8. Long Lineage Problem

- If the lineage chain is very long, recomputation becomes slow and expensive.
- This increases the risk of job failure.
- **Solution:** Use checkpoint() or persist() wisely.

Example of Long Lineage:



Too long! Recomputing takes time.

Key Point

Break long lineage using checkpoint() to improve reliability and speed.

9. Common Operations and Dependency Type

Operation	Dependency Type	Needs Shuffle?
map(), filter(), flatMap()	Narrow	No
union(), sample()	Narrow	No
groupByKey(), reduceByKey()	Wide	Yes
join(), cogroup()	Wide	Yes
distinct(), sortByKey()	Wide	Yes
coalesce()	Narrow	No
repartition()	Wide	Yes

Key Point

Wide dependency = Shuffle = More cost.

10. Quick Revision

- ✅ Lineage is the history of transformations.
- ✅ RDDs are immutable; lineage ensures fault tolerance.
- ✅ Narrow dependency = One-to-one → No shuffle.
- ✅ Wide dependency = Many-to-many → Shuffle required.
- ✅ Lineage helps Spark recompute lost partitions.
- ✅ Use checkpoint() to break long lineage.

11. Interview Questions

1. What is RDD lineage in Spark?
2. How does Spark achieve fault tolerance?
3. What is the difference between narrow and wide dependency?
4. Give examples of narrow and wide transformations.
5. How does lineage help in case of partition loss?
6. What is the difference between lineage and checkpoint?
7. Why is long lineage a problem?
8. When should we use checkpoint()?
9. Does map() cause a shuffle? Why or why not?
10. How does Spark rebuild a lost partition?

12. Memory Trick

- **L** – Log of transformations
- **I** – Immutable RDDs
- **N** – Narrow or Wide dependency
- **E** – Ensures fault tolerance
- **A** – Allows recomputation
- **G** – Guarantees reliability
- **E** – Efficient processing

LINEAGE
Remembers the journey, so Spark can rebuild the destination!

RDD Lineage = The History that Makes Spark Resilient, Reliable and Recoverable!